

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота 3
з дисципліни
«Об'єктно-орієнтоване програмування»
на тему
« Розробка інтерфейсу користувача »

Виконав:

студент групи ІМ-55

Бабенко Ярослав Володимирович

Номер в списку - 3

Перевірив:

Рекечинський Д.О.

Київ 2026

Завдання

1. Створити у середовищі розробки проект з ім'ям Lab3.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт

Варіант завдання (Лабораторна робота №3)

Оскільки для попередньої лабораторної роботи номер за журналом був $J = 3$, для вибору варіанту поточної роботи використовується значення $J = J_{\text{лаб2}} + 1$, отже розрахунковий номер варіанту **$J = 4$** . Згідно з методичними вказівками, вимоги щодо фігур повторюються із новим значенням J .

1. Індивідуальні вимоги до малювання фігур ($J = 4$):

- **Сховище об'єктів:** Статичний масив вказівників `Shape *pcshape[104]`; (оскільки $4 \bmod 3 = 1$, а розмір $N = 4 + 100 = 104$).
- **"Гумовий" слід при вводі об'єктів:** Суцільна лінія чорного кольору (оскільки $4 \bmod 4 = 0$).
- **Прямокутник:**
 - Ввід по двох протилежних кутах (оскільки $4 \bmod 2 = 0$).
 - Відображення: чорний контур прямокутника без заповнення (оскільки $4 \bmod 5 = 4$).
- **Еліпс:**
 - Ввід від центру до одного з кутів охоплюючого прямокутника (оскільки $4 \bmod 2 = 0$).
 - Відображення: чорний контур з рожевим заповненням (оскільки $4 \bmod 5 = 4$; колір — $4 \bmod 6 = 4$).
- **Позначка поточного типу об'єкту:** Здійснюється в меню "Об'єкти" через метод `OnInitMenuPopup` (оскільки $4 \bmod 2 = 0$).

2. Вимоги до об'єктно-орієнтованої архітектури вікна:

- Усі методи-обробники повідомлень (зокрема, і метод `OnNotify`) повинні бути функціями-членами спеціально розробленого класу вікна (наприклад, `MyEditor`).

- У звіті обов'язково повинна бути наведена схема успадкування класів — діаграма класів.

3. Вимоги до інтерфейсу користувача (Toolbar та меню):

- У головному вікні повинно бути вікно Toolbar з кнопками, що відповідають типам об'єктів (Крапка, Лінія, Прямокутник, Еліпс) та дублюють підпункти меню "Об'єкти".
- Кнопки на панелі Toolbar повинні бути не з текстовим підписом, а з іконкою, за якою можна здогадатися про інструмент.
- Кнопки панелі Toolbar обов'язково мають бути забезпечені спливаючими підказками (tooltips).
- Меню "Об'єкти" повинно розташовуватися праворуч меню "Файл" та ліворуч меню "Довідка".
- Підпункти меню "Об'єкти" повинні містити назви геометричних форм українською мовою.

Вихідний текст файлів:

editor.cpp:

```
#include "editor.h"
```

```
MyEditor::MyEditor(HWND h) : hWnd(h), shapeCount(0), currentTool(0), isDrawing(false), tempShape(nullptr),
hRubberPen(nullptr) {
for (int i = 0; i < MAX_SHAPES; i++) {
    pcshape[i] = nullptr;
}
}
```

```
MyEditor::~MyEditor() {
    for (int i = 0; i < shapeCount; i++) {
        if (pcshape[i]) delete pcshape[i];
    }
    if (tempShape) delete tempShape;
    if (hRubberPen) DeleteObject(hRubberPen);
}
```

```
Shape* MyEditor::CreateShapeFactory(int tool) {
    switch (tool) {
        case 0: return new PointShape();
        case 1: return new LineShape();
        case 2: return new RectShape();
        case 3: return new EllipseShape();
    }
    return nullptr;
}
```

```

void MyEditor::CreateToolBar() {
    InitCommonControls();

    TBBUTTON tbb[4];
    ZeroMemory(tbb, sizeof(tbb));

    tbb[0].iBitmap = STD_FILENEW; tbb[0].idCommand = ID_OBJECT_POINT; tbb[0].fsState = TBSTATE_ENABLED | TBSTATE_CHECKED; tbb[0].fsStyle = TBSTYLE_CHECKGROUP;
    tbb[1].iBitmap = STD_FILEOPEN; tbb[1].idCommand = ID_OBJECT_LINE; tbb[1].fsState = TBSTATE_ENABLED; tbb[1].fsStyle = TBSTYLE_CHECKGROUP;
    tbb[2].iBitmap = STD_FILESAVE; tbb[2].idCommand = ID_OBJECT_RECT; tbb[2].fsState = TBSTATE_ENABLED; tbb[2].fsStyle = TBSTYLE_CHECKGROUP;
    tbb[3].iBitmap = STD_PRINT; tbb[3].idCommand = ID_OBJECT_ELLIPSE; tbb[3].fsState = TBSTATE_ENABLED; tbb[3].fsStyle = TBSTYLE_CHECKGROUP;

    hToolBar = CreateToolBarEx(hWnd,
        WS_CHILD | WS_VISIBLE | WS_BORDER | TBSTYLE_TOOLTIPS,
        1001, 4, HINST_COMMCTRL, IDB_STD_SMALL_COLOR,
        tbb, 4, 0, 0, 0, 0, sizeof(TBBUTTON));
}

void MyEditor::OnLButtonDown(LPARAM lParam) {
    if (shapeCount < MAX_SHAPES) {
        isDrawing = true;
        startX = endX = LOWORD(lParam);
        startY = endY = HIWORD(lParam);
        tempShape = CreateShapeFactory(currentTool);
        hRubberPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    }
}

void MyEditor::OnMouseMove(LPARAM lParam) {
    if (isDrawing && tempShape) {
        HDC hdc = GetDC(hWnd);
        SetROP2(hdc, R2_NOTXORPEN);
        SetBkMode(hdc, TRANSPARENT);

        HPEN oldPen = (HPEN)SelectObject(hdc, hRubberPen);

        tempShape->Set(startX, startY, endX, endY);
        tempShape->Show(hdc);

        endX = LOWORD(lParam);
        endY = HIWORD(lParam);

        tempShape->Set(startX, startY, endX, endY);
        tempShape->Show(hdc);

        SelectObject(hdc, oldPen);
        ReleaseDC(hWnd, hdc);
    }
}

```

```
}
```

```
void MyEditor::OnLButtonUp(LPARAM lParam) {  
    if (isDrawing) {  
        isDrawing = false;  
        pcshape[shapeCount] = CreateShapeFactory(currentTool);  
        pcshape[shapeCount]->Set(startX, startY, endX, endY);  
        shapeCount++;  
    }
```

```
    delete tempShape;  
    tempShape = nullptr;
```

```
    if (hRubberPen) {  
        DeleteObject(hRubberPen);  
        hRubberPen = nullptr;  
    }  
    InvalidateRect(hWnd, NULL, TRUE);  
}
```

```
}
```

```
void MyEditor::OnPaint() {  
    PAINTSTRUCT ps;  
    HDC hdc = BeginPaint(hWnd, &ps);  
    for (int i = 0; i < shapeCount; i++) {  
        if (pcshape[i]) pcshape[i]->Show(hdc);  
    }  
    EndPaint(hWnd, &ps);  
}
```

```
void MyEditor::OnCommand(WPARAM wParam) {  
    int wmId = LOWORD(wParam);  
    switch (wmId) {  
        case ID_OBJECT_POINT: currentTool = 0; break;  
        case ID_OBJECT_LINE: currentTool = 1; break;  
        case ID_OBJECT_RECT: currentTool = 2; break;  
        case ID_OBJECT_ELLIPSE: currentTool = 3; break;  
    }
```

```
    if (hToolBar) {  
        SendMessage(hToolBar, TB_CHECKBUTTON, ID_OBJECT_POINT + currentTool, MAKELONG(TRUE, 0));  
    }  
}
```

```
void MyEditor::OnInitMenuPopup(WPARAM wParam, LPARAM lParam) {  
    HMENU hMenu = (HMENU)wParam;
```

```
    CheckMenuItem(hMenu, ID_OBJECT_POINT, MF_BYCOMMAND | MF_UNCHECKED);  
    CheckMenuItem(hMenu, ID_OBJECT_LINE, MF_BYCOMMAND | MF_UNCHECKED);  
    CheckMenuItem(hMenu, ID_OBJECT_RECT, MF_BYCOMMAND | MF_UNCHECKED);  
    CheckMenuItem(hMenu, ID_OBJECT_ELLIPSE, MF_BYCOMMAND | MF_UNCHECKED);
```

```
    CheckMenuItem(hMenu, ID_OBJECT_POINT + currentTool, MF_BYCOMMAND | MF_CHECKED);  
}
```

```

LRESULT MyEditor::OnNotify(LPPARAM lParam) {
    LPNMHDR pnmh = (LPNMHDR)lParam;

    if (pnmh->code == TTN_GETDISPINFOW) {
        LPTOOLTIPTEXTW lpttt = (LPTOOLTIPTEXTW)lParam;
        lpttt->hinst = NULL;

        static WCHAR strPoint[] = { 0x041A, 0x0440, 0x0430, 0x043F, 0x043A, 0x0430, 0 }; // Крапка
        static WCHAR strLine[] = { 0x041B, 0x0456, 0x043D, 0x0456, 0x044F, 0 }; // Лінія
        static WCHAR strRect[] = { 0x041F, 0x0440, 0x044F, 0x043C, 0x043E, 0x043A, 0x0443, 0x0442, 0x043D, 0x0438,
0x043A, 0 }; // Прямокутник
        static WCHAR strEllipse[] = { 0x0415, 0x043B, 0x0456, 0x043F, 0x0441, 0 }; // Еліпс

        switch (lpttt->hdr.idFrom) {
            case ID_OBJECT_POINT: lpttt->lpszText = strPoint; break;
            case ID_OBJECT_LINE: lpttt->lpszText = strLine; break;
            case ID_OBJECT_RECT: lpttt->lpszText = strRect; break;
            case ID_OBJECT_ELLIPSE: lpttt->lpszText = strEllipse; break;
        }
    }
    else if (pnmh->code == TTN_GETDISPINFOA) {
        LPTOOLTIPTEXTA lpttt = (LPTOOLTIPTEXTA)lParam;
        lpttt->hinst = NULL;

        static char strPointA[] = { (char)0xCA, (char)0xF0, (char)0xE0, (char)0xEF, (char)0xEA, (char)0xE0, 0 };
        static char strLineA[] = { (char)0xCB, (char)0xB3, (char)0xED, (char)0xB3, (char)0xFF, 0 };
        static char strRectA[] = { (char)0xCF, (char)0xF0, (char)0xFF, (char)0xEC, (char)0xEE, (char)0xEA, (char)0xF3,
(char)0xF2, (char)0xED, (char)0xE8, (char)0xEA, 0 };
        static char strEllipseA[] = { (char)0xC5, (char)0xEB, (char)0xB3, (char)0xEF, (char)0xF1, 0 };

        switch (lpttt->hdr.idFrom) {
            case ID_OBJECT_POINT: lpttt->lpszText = strPointA; break;
            case ID_OBJECT_LINE: lpttt->lpszText = strLineA; break;
            case ID_OBJECT_RECT: lpttt->lpszText = strRectA; break;
            case ID_OBJECT_ELLIPSE: lpttt->lpszText = strEllipseA; break;
        }
    }
    return 0;
}

```

shape.cpp:

```
#include "editor.h"
```

```

MyEditor::MyEditor(HWND h) : hWnd(h), shapeCount(0), currentTool(0), isDrawing(false), tempShape(nullptr),
hRubberPen(nullptr) {
    for (int i = 0; i < MAX_SHAPES; i++) {
        pcshape[i] = nullptr;
    }
}

```

```

MyEditor::~MyEditor() {
    for (int i = 0; i < shapeCount; i++) {

```

```

        if (pcshape[i]) delete pcshape[i];
    }
    if (tempShape) delete tempShape;
    if (hRubberPen) DeleteObject(hRubberPen);
}

```

```

Shape* MyEditor::CreateShapeFactory(int tool) {
    switch (tool) {
        case 0: return new PointShape();
        case 1: return new LineShape();
        case 2: return new RectShape();
        case 3: return new EllipseShape();
    }
    return nullptr;
}

```

```

void MyEditor::CreateToolbar() {
    InitCommonControls();

```

```

    TBBUTTON tbb[4];
    ZeroMemory(tbb, sizeof(tbb));

```

```

    tbb[0].iBitmap = STD_FILENEW; tbb[0].idCommand = ID_OBJECT_POINT; tbb[0].fsState = TBSTATE_ENABLED |
TBSTATE_CHECKED; tbb[0].fsStyle = TBSTYLE_CHECKGROUP;
    tbb[1].iBitmap = STD_FILEOPEN; tbb[1].idCommand = ID_OBJECT_LINE; tbb[1].fsState = TBSTATE_ENABLED;
tbb[1].fsStyle = TBSTYLE_CHECKGROUP;
    tbb[2].iBitmap = STD_FILESAVE; tbb[2].idCommand = ID_OBJECT_RECT; tbb[2].fsState = TBSTATE_ENABLED;
tbb[2].fsStyle = TBSTYLE_CHECKGROUP;
    tbb[3].iBitmap = STD_PRINT; tbb[3].idCommand = ID_OBJECT_ELLIPSE; tbb[3].fsState = TBSTATE_ENABLED;
tbb[3].fsStyle = TBSTYLE_CHECKGROUP;

```

```

    hToolBar = CreateToolBarEx(hWnd,
        WS_CHILD | WS_VISIBLE | WS_BORDER | TBSTYLE_TOOLTIPS,
        1001, 4, HINST_COMMCTRL, IDB_STD_SMALL_COLOR,
        tbb, 4, 0, 0, 0, 0, sizeof(TBBUTTON));
}

```

```

void MyEditor::OnLButtonDown(LPARAM lParam) {
    if (shapeCount < MAX_SHAPES) {
        isDrawing = true;
        startX = endX = LOWORD(lParam);
        startY = endY = HIWORD(lParam);
        tempShape = CreateShapeFactory(currentTool);
        hRubberPen = CreatePen(PS_SOLID, 1, RGB(0, 0, 0));
    }
}

```

```

void MyEditor::OnMouseMove(LPARAM lParam) {
    if (isDrawing && tempShape) {
        HDC hdc = GetDC(hWnd);
        SetROP2(hdc, R2_NOTXORPEN);
        SetBkMode(hdc, TRANSPARENT);
    }
}

```

```

    HPEN oldPen = (HPEN)SelectObject(hdc, hRubberPen);

    tempShape->Set(startX, startY, endX, endY);
    tempShape->Show(hdc);

    endX = LOWORD(lParam);
    endY = HIWORD(lParam);

    tempShape->Set(startX, startY, endX, endY);
    tempShape->Show(hdc);

    SelectObject(hdc, oldPen);
    ReleaseDC(hWnd, hdc);
}
}

void MyEditor::OnLButtonUp(LPARAM lParam) {
    if (isDrawing) {
        isDrawing = false;
        pcshape[shapeCount] = CreateShapeFactory(currentTool);
        pcshape[shapeCount]->Set(startX, startY, endX, endY);
        shapeCount++;

        delete tempShape;
        tempShape = nullptr;

        if (hRubberPen) {
            DeleteObject(hRubberPen);
            hRubberPen = nullptr;
        }
        InvalidateRect(hWnd, NULL, TRUE);
    }
}

void MyEditor::OnPaint() {
    PAINTSTRUCT ps;
    HDC hdc = BeginPaint(hWnd, &ps);
    for (int i = 0; i < shapeCount; i++) {
        if (pcshape[i]) pcshape[i]->Show(hdc);
    }
    EndPaint(hWnd, &ps);
}

void MyEditor::OnCommand(WPARAM wParam) {
    int wmId = LOWORD(wParam);
    switch (wmId) {
        case ID_OBJECT_POINT: currentTool = 0; break;
        case ID_OBJECT_LINE: currentTool = 1; break;
        case ID_OBJECT_RECT: currentTool = 2; break;
        case ID_OBJECT_ELLIPSE: currentTool = 3; break;
    }

    if (hToolBar) {

```

```

        SendMessage(hToolBar, TB_CHECKBUTTON, ID_OBJECT_POINT + currentTool, MAKELONG(TRUE, 0));
    }
}

void MyEditor::OnInitMenuPopup(WPARAM wParam, LPARAM lParam) {
    HMENU hMenu = (HMENU)wParam;

    CheckMenuItem(hMenu, ID_OBJECT_POINT, MF_BYCOMMAND | MF_UNCHECKED);
    CheckMenuItem(hMenu, ID_OBJECT_LINE, MF_BYCOMMAND | MF_UNCHECKED);
    CheckMenuItem(hMenu, ID_OBJECT_RECT, MF_BYCOMMAND | MF_UNCHECKED);
    CheckMenuItem(hMenu, ID_OBJECT_ELLIPSE, MF_BYCOMMAND | MF_UNCHECKED);

    CheckMenuItem(hMenu, ID_OBJECT_POINT + currentTool, MF_BYCOMMAND | MF_CHECKED);
}

LRESULT MyEditor::OnNotify(LPARAM lParam) {
    LPNMHDR pnmh = (LPNMHDR)lParam;

    if (pnmh->code == TTN_GETDISPINFOW) {
        LPTOOLTIPTEXTW lpttt = (LPTOOLTIPTEXTW)lParam;
        lpttt->hinst = NULL;

        static WCHAR strPoint[] = { 0x041A, 0x0440, 0x0430, 0x043F, 0x043A, 0x0430, 0 }; // Крапка
        static WCHAR strLine[] = { 0x041B, 0x0456, 0x043D, 0x0456, 0x044F, 0 }; // Лінія
        static WCHAR strRect[] = { 0x041F, 0x0440, 0x044F, 0x043C, 0x043E, 0x043A, 0x0443, 0x0442, 0x043D, 0x0438,
0x043A, 0 }; // Прямокутник
        static WCHAR strEllipse[] = { 0x0415, 0x043B, 0x0456, 0x043F, 0x0441, 0 }; // Еліпс

        switch (lpttt->hdr.idFrom) {
            case ID_OBJECT_POINT: lpttt->lpszText = strPoint; break;
            case ID_OBJECT_LINE: lpttt->lpszText = strLine; break;
            case ID_OBJECT_RECT: lpttt->lpszText = strRect; break;
            case ID_OBJECT_ELLIPSE: lpttt->lpszText = strEllipse; break;
        }
    }
    else if (pnmh->code == TTN_GETDISPINFOA) {
        LPTOOLTIPTEXTA lpttt = (LPTOOLTIPTEXTA)lParam;
        lpttt->hinst = NULL;

        static char strPointA[] = { (char)0xCA, (char)0xF0, (char)0xE0, (char)0xEF, (char)0xEA, (char)0xE0, 0 };
        static char strLineA[] = { (char)0xCB, (char)0xB3, (char)0xED, (char)0xB3, (char)0xFF, 0 };
        static char strRectA[] = { (char)0xCF, (char)0xF0, (char)0xFF, (char)0xEC, (char)0xEE, (char)0xEA, (char)0xF3,
(char)0xF2, (char)0xED, (char)0xE8, (char)0xEA, 0 };
        static char strEllipseA[] = { (char)0xC5, (char)0xEB, (char)0xB3, (char)0xEF, (char)0xF1, 0 };

        switch (lpttt->hdr.idFrom) {
            case ID_OBJECT_POINT: lpttt->lpszText = strPointA; break;
            case ID_OBJECT_LINE: lpttt->lpszText = strLineA; break;
            case ID_OBJECT_RECT: lpttt->lpszText = strRectA; break;
            case ID_OBJECT_ELLIPSE: lpttt->lpszText = strEllipseA; break;
        }
    }
}

```

```
    return 0;
}
```

Lab3.cpp:

```
#include "framework.h"
#include "resource.h"
#include "editor.h"
```

```
#pragma comment(lib, "comctl32.lib")
```

```
HINSTANCE hInst;
WCHAR szTitle[100] = L"Lab3 - Інтерфейс користувача";
WCHAR szWindowClass[100] = L"Lab3Class";
```

```
ATOM MyRegisterClass(HINSTANCE hInstance);
BOOL InitInstance(HINSTANCE, int);
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
```

```
int APIENTRY wWinMain(_In_ HINSTANCE hInstance, _In_opt_ HINSTANCE hPrevInstance, _In_ LPWSTR lpCmdLine,
_In_int nCmdShow) {
    MyRegisterClass(hInstance);
    if (!InitInstance(hInstance, nCmdShow)) return FALSE;
```

```
    MSG msg;
    while (GetMessage(&msg, nullptr, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    return (int)msg.wParam;
}
```

```
ATOM MyRegisterClass(HINSTANCE hInstance) {
    WNDCLASSEXW wcex;
    wcex.cbSize = sizeof(WNDCLASSEX);
    wcex.style = CS_HREDRAW | CS_VREDRAW;
    wcex.lpfnWndProc = WndProc;
    wcex.cbClsExtra = 0;
    wcex.cbWndExtra = 0;
    wcex.hInstance = hInstance;
    wcex.hIcon = LoadIcon(nullptr, IDI_APPLICATION);
    wcex.hCursor = LoadCursor(nullptr, IDC_ARROW);
    wcex.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

    wcex.lpszMenuName = MAKEINTRESOURCEW(IDC_LAB3);

    wcex.lpszClassName = szWindowClass;
    wcex.hIconSm = LoadIcon(nullptr, IDI_APPLICATION);
    return RegisterClassExW(&wcex);
}
```

```
BOOL InitInstance(HINSTANCE hInstance, int nCmdShow) {
    hInst = hInstance;
```

```

HWND hWnd = CreateWindowW(szWindowClass, szTitle, WS_OVERLAPPEDWINDOW,
    CW_USEDEFAULT, 0, CW_USEDEFAULT, 0, nullptr, nullptr, hInstance, nullptr);
if (!hWnd) return FALSE;
ShowWindow(hWnd, nCmdShow);
UpdateWindow(hWnd);
return TRUE;
}

LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM wParam, LPARAM lParam) {

    MyEditor* editor = (MyEditor*)GetWindowLongPtr(hWnd, GWLP_USERDATA);

    switch (message) {
    case WM_CREATE:
        editor = new MyEditor(hWnd);
        SetWindowLongPtr(hWnd, GWLP_USERDATA, (LONG_PTR)editor);
        editor->CreateToolbar();
        break;

    case WM_LBUTTONDOWN: if (editor) editor->OnLButtonDown(lParam); break;
    case WM_MOUSEMOVE:   if (editor) editor->OnMouseMove(lParam); break;
    case WM_LBUTTONUP:   if (editor) editor->OnLButtonUp(lParam); break;
    case WM_PAINT:       if (editor) editor->OnPaint(); break;

    case WM_COMMAND:
        if (editor) editor->OnCommand(wParam);
        if (LOWORD(wParam) == 105) DestroyWindow(hWnd);
        break;

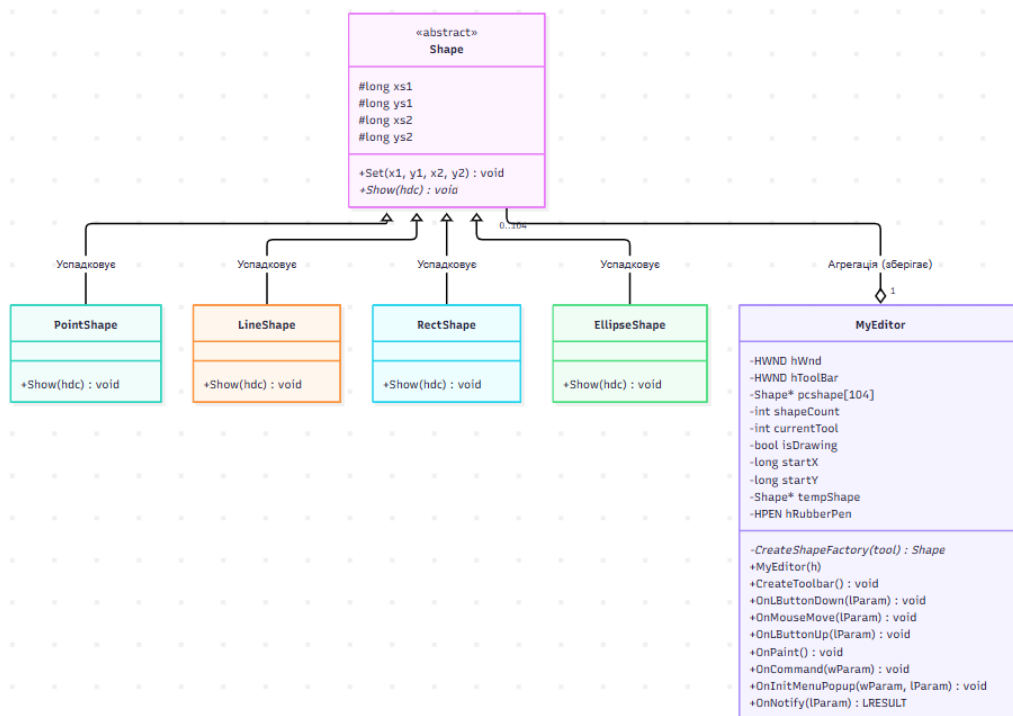
    case WM_INITMENUPOPUP: if (editor) editor->OnInitMenuPopup(wParam, lParam); break;
    case WM_NOTIFY:        if (editor) return editor->OnNotify(lParam); break;

    case WM_DESTROY:
        if (editor) delete editor;
        PostQuitMessage(0);
        break;

    default:
        return DefWindowProc(hWnd, message, wParam, lParam);
    }
    return 0;
}

```

Діаграма класів:



Висновок:

Я отримав вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши графічний інтерфейс користувача.